

1292. Maximum Side Length of a Square with Sum Less than or Equal to Threshold

Solved 

Medium

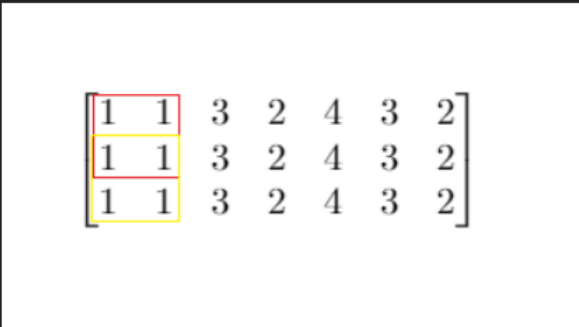
 Topics

 Companies

 Hint

Given a $m \times n$ matrix `mat` and an integer `threshold`, return *the maximum side-length of a square with a sum less than or equal to `threshold`* or return `0` if there is no such square.

Example 1:



1	1	3	2	4	3	2
1	1	3	2	4	3	2
1	1	3	2	4	3	2

Input: `mat = [[1,1,3,2,4,3,2],[1,1,3,2,4,3,2],[1,1,3,2,4,3,2]]`, `threshold = 4`

Output: `2`

Explanation: The maximum side length of square with sum less than 4 is 2 as shown.

Example 2:

Input: `mat = [[2,2,2,2,2],[2,2,2,2,2],[2,2,2,2,2],[2,2,2,2,2],[2,2,2,2,2]]`,
`threshold = 1`

Output: `0`

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 300`
- `0 <= mat[i][j] <= 104`
- `0 <= threshold <= 105`

Python:

class Solution:

```

def maxSideLength(self, mat: List[List[int]], threshold: int) -> int:
    m, n = len(mat), len(mat[0])

    pref = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            pref[i][j] = (
                mat[i-1][j-1]
                + pref[i-1][j]
                + pref[i][j-1]
                - pref[i-1][j-1]
            )

    maxSide = min(m, n)

    while maxSide > 0:
        for i in range(m - maxSide + 1):
            for j in range(n - maxSide + 1):
                s = (
                    pref[i+maxSide][j+maxSide]
                    - pref[i][j+maxSide]
                    - pref[i+maxSide][j]
                    + pref[i][j]
                )
                if s <= threshold:
                    return maxSide
            maxSide -= 1

    return 0

```

JavaScript:

```

var maxSideLength = function(mat, threshold) {
    const m = mat.length, n = mat[0].length;

    const pref = Array.from({ length: m + 1 }, () =>
        Array(n + 1).fill(0)
    );

    for (let i = 1; i <= m; i++) {
        for (let j = 1; j <= n; j++) {
            pref[i][j] =
                mat[i-1][j-1]
                + pref[i-1][j]

```

```

        + pref[i][j-1]
        - pref[i-1][j-1];
    }
}

let maxSide = Math.min(m, n);

while (maxSide > 0) {
    for (let i = 0; i + maxSide <= m; i++) {
        for (let j = 0; j + maxSide <= n; j++) {
            const sum =
                pref[i+maxSide][j+maxSide]
                - pref[i][j+maxSide]
                - pref[i+maxSide][j]
                + pref[i][j];
            if (sum <= threshold) return maxSide;
        }
    }
    maxSide--;
}
return 0;
};

```

Java:

```

class Solution {
    public int maxSideLength(int[][] mat, int threshold) {
        int m = mat.length, n = mat[0].length;
        int maxSide = Math.min(m,n);

        int[][] pref = new int[m+1][n+1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                pref[i][j] =
                    mat[i-1][j-1]
                    + pref[i-1][j]
                    + pref[i][j-1]
                    - pref[i-1][j-1];
            }
        }

        while (maxSide > 0) {
            for (int i = 0; i + maxSide <= m; i++) {
                for (int j = 0; j + maxSide <= n; j++) {
                    if (helper(pref, threshold, i, j, maxSide))

```

```
        return maxSide;
    }
}
maxSide--;
}

return 0;
}
```

```
private boolean helper(int[][] pref, int t, int x, int y, int side) {
    int x2 = x + side;
    int y2 = y + side;

    int sum =
        pref[x2][y2]
        - pref[x][y2]
        - pref[x2][y]
        + pref[x][y];

    return sum <= t;
}
}
```